IVS - profiling

Segedínsky Guláš

26. apríla 2026

1 Úvod

Predmetom analýzy je overenie rýchlosti výpočtu smerodajnej odchýlky v programe `stddev.py`. Testovanie prebehlo na platformách Linux a WSL s cieľom odhaliť kritické časti kódu, ktoré spomaľujú matematickú knižnicu projektu „Kalkulačka“. Výsledky profilovania definujú smer, ktorým by sa mala uberať následná optimalizácia softvéru.

2 Profiling

Profilovanie sme realizovali na osobnom počítači. Použili sme štandardný analytický modul `cProfile`, ktorý pracuje na báze merania času stráveného v jednotlivých volaniach funkcií. Ako testovacie dáta sme zvolili vstup 10^6 náhodne vygenerovaných desatinných čísel typu float v rozsahu od 1 do 1000.

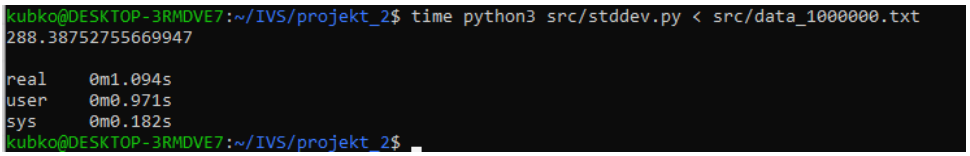
Samotný proces profilovania sme spustili príkazom:

```
python3 -m cProfile -s cumulative stddev.py < data.txt > vystup.txt
```

Týmto spôsobom interpret Pythonu vygeneruje štatistiku volaní, zoradí ich podľa kumulatívneho času a výsledky uloží do súboru `vystup.txt`, ktorý slúži ako podklad pre ďalšiu analýzu.

3 Analýza

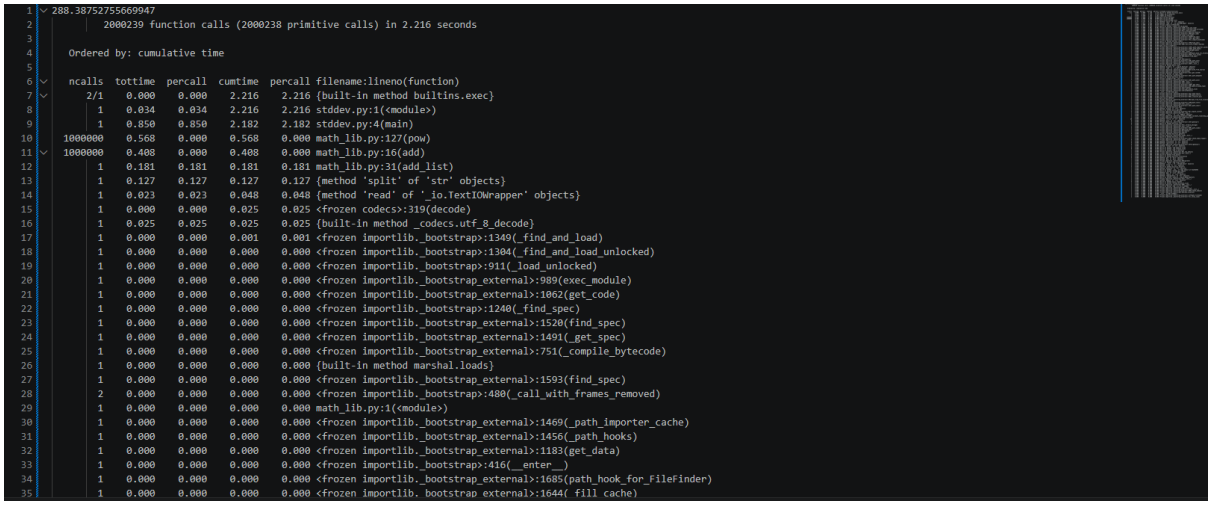
Pre získanie referenčných hodnôt a zistenie reálnej réžie profilovacieho nástroja sme program najprv spustili čisto pomocou systémového príkazu `time`. Pri testovaní v prostredí WSL dosiahol celkový čas behu programu (real time) približne 1,1 sekundy. Pri natívnom spustení na operačnom systéme Linux bol čas vykonania ešte približne o 0,2 sekundy kratší (približne 0,9 sekundy).



Obr. 1: Spustenie programu bez profileru pomocou príkazu time (WSL)

Získané dáta z následného profilovania nám poskytujú detailný prehľad o časovej náročnosti jednotlivých funkcií. Z vygenerovaného výstupu modulu `cProfile` vidíme, že pri zapnutom profilovaní trvalo spracovanie 10^6 hodnôt celkovo 2,216 sekundy. V porovnaní s bežným spustením (1,1 s) je teda zrejmé, že samotné profilovanie spomaľuje beh programu zhruba na dvojnásobok.

Z analýzy nameraných hodnôt je zrejmé, že okrem samotného načítavania a réžie hlavnej funkcie `main` vznikajú najväčšie časové straty v matematických operáciách našej knižnice `math_lib.py`. Konkrétne ide o funkcie `pow` (umocňovanie) a `add` (sčítanie). Funkcia `pow` bola v cykle zavolaná presne 1 000 000-krát, pričom jej kumulatívny čas dosiahol 0,568 sekundy. Hneď za ňou nasleduje funkcia `add` s rovnakým počtom volaní a časom 0,408 sekundy.



```
288.38752755669947
         2000239 function calls (2000238 primitive calls) in 2.216 seconds

   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      2/1    0.000    0.000    2.216    2.216 {built-in method builtins.exec}
        1    0.034    0.034    2.216    2.216 stddev.py:1(<module>)
        1    0.850    0.850    2.182    2.182 stddev.py:4(main)
  1000000    0.568    0.000    0.568    0.000 math_lib.py:127(pow)
  1000000    0.408    0.000    0.408    0.000 math_lib.py:16(add)
        1    0.181    0.181    0.181    0.181 math_lib.py:31(add_list)
        1    0.127    0.127    0.127    0.127 {method 'split' of 'str' objects}
        1    0.023    0.023    0.048    0.048 {method 'read' of '_io.TextIOWrapper' objects}
        1    0.000    0.000    0.025    0.025 <frozen codecs>:319(decode)
        1    0.025    0.025    0.025    0.025 {built-in method _codecs.utf_8_decode}
        1    0.000    0.000    0.001    0.001 <frozen importlib._bootstrap>:1349(_find_and_load)
        1    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:1304(_find_and_load_unlocked)
        1    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:911(_load_unlocked)
        1    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap_external>:989(exec_module)
        1    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap_external>:1062(get_code)
        1    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:1240(_find_spec)
        1    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap_external>:1520(find_spec)
        1    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap_external>:1491(_get_spec)
        1    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap_external>:751(_compile_bytecode)
        1    0.000    0.000    0.000    0.000 {built-in method marshal.loads}
        1    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap_external>:1593(find_spec)
        2    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:480(_call_with_frames_removed)
        1    0.000    0.000    0.000    0.000 math_lib.py:1(<module>)
        1    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap_external>:1469(_path_importer_cache)
        1    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap_external>:1456(_path_hooks)
        1    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap_external>:1183(get_data)
        1    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:416(__enter__)
        1    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap_external>:1685(path_hook_for_FileFinder)
        1    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap_external>:1644(_fill_cache)
```

Obr. 2: Výstup profilovania modulu cProfile zoradený podľa kumulatívneho času

4 Záver

Profilovanie preukázalo, že program strávi najvýraznejšiu časť výpočtového času opakovaným volaním elementárnych matematických operácií z knižnice `math_lib.py`. Najväčšiu záťaž predstavujú funkcie `pow` a `add`, ktoré sú pri výpočte smerodajnej odchýlky volané iteratívne pre každý jeden prvok rozsiahleho zoznamu. Namerané výsledky taktiež ukázali očakávaný rozdiel vo výkone medzi prostredím WSL a natívnym Linuxom (v prospech Linuxu o cca 0,2 s), a rovnako aj réžiu samotného profileru, ktorý predĺžil beh o 1 sekundu. Pre razantné zlepšenie výkonu by bolo vhodné matematické funkcie zoptimalizovať, prípadne nahradiť iteratívne spracovanie v cykle efektívnejšími operáciami.

3